

Retention Time Tests on Gain Cell Embedded Dynamic Random Access Memory

Hannah Bialik

Advised under Prof. Adam Teman and PhD student Odem Harel

Computers require the ability to store information to harness user data effectively. Memory embedded in a microchip transmits data more quickly and at lower power costs than external memories. Typically, Static Random Access Memory (SRAM) is utilized as embedded RAM because it is quick and reliable. However, with each bit-cell requiring 6 transistors, the storage density of SRAM is quite low. Embedded Dynamic Random Access Memory (eDRAM) macrocells have been developed as a higher density alternative to SRAM. The traditional version of eDRAM (1T-1C) is built with one single-access transistor and a special high-density 3D capacitor. The atypical manufacturing process of the high-density capacitor makes the 1T-1C eDRAM impractical in many situations. Gain Cell eDRAM (GC-eDRAM) is an alternative form of eDRAM with a basic bit-cell of 2-4 MOS transistors that can be easily integrated into a chip. As with all types of dynamic memory, data is stored as electric charge on a capacitor. Therefore, leakage currents compromise the stored data after some time, creating the necessity for periodic refresh operations to ensure that the data is not lost. The main downside of GC-eDRAM is its low retention time, which results in higher power consumption due to more frequent data refresh cycles. The Emerging Nanoscaled Integrated Circuit and Systems (EnICS) lab is developing methods of extending the refresh period - the length of time between refresh operations without losing the data - so that GC-eDRAM can be a more practical eDRAM option.

My main role at EnICS has been to test the data retention time of the GC-eDRAM on the Lupulus and Leo2 chips and to create graphical representations of that information. This involved writing, editing, and combining C and Python codes that interacted with the chips and a DC voltage supply. I also manually read and adjusted voltages on the research board that connects to the chip. In a most basic bit retention time test, a pattern of binary bits is written to an area of the chip in the form of an N-bit word. There is then an idle period after which the stored data is read. If there is a mismatch between what was previously written and what is read after the idle period, that means some bits failed. The retention time of a bit is the length of time it takes until the first time it fails, meaning that it flips (from 0 to 1 or vice versa) from the value that was originally written. However, since we can only write and read to a word at a time (32 bits in the case of the Leo2 and Lupulus), the read data must be parsed and bits that flipped for the first time in this reading must be identified. The retention time of these bits is equivalent to the idle period between writing and reading the word. After these calculations, if not all bits in the word have failed once, the word is written again with an increased idle period. Bits that fail for the first time in that round have a retention time of the increased idle period. This cycle continues until all bits in the word have failed at least once. The flowchart in Figure 1 illustrates this process.

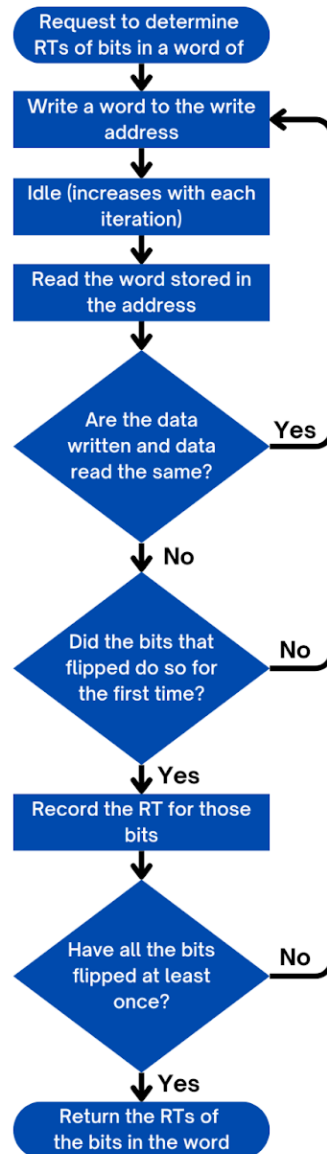


Figure 1. Flow chart of Bit Retention Time (RT) code.

Once this data was collected, I produced heatmaps like the one in Figure 2. Each square represents a gain cell which stores a bit of information. The Data Retention Time (DRT) of that bit is correlated to the square's color. The retention times in the figure are determined based on writing all ones to the bits, which is the worst-case scenario since ones have a lower retention time than zeros due to hardware design decisions. (This means writing 0xffffffff to each 32-bit word.) While there is some variation in the retention time of cells in this sample, they are overall similar. This indicates relative uniformity among the transistors.

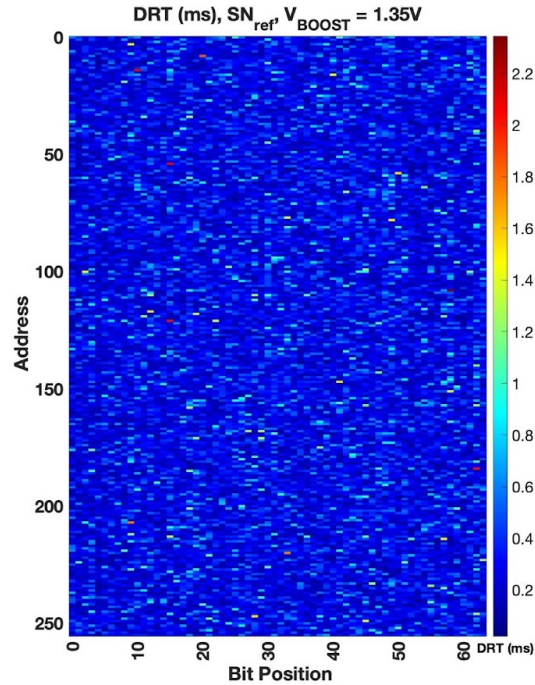


Figure 2. Heatmap of Data Retention Time of 2T Gain Cells in the Leo2 Chip.

Inspired by concepts in Leo2 code (since for that chip I modified code that I had received), I set out to write the code to test the Lupulus retention time. Since it is tedious to change the input voltages with a screwdriver on the test board (and we wanted to test the retention time with various voltage inputs), we planned to hook the board up to an external DC power source and use the code to manipulate the input voltages. After some struggles accessing the correct location on the chip, my code worked and properly connected to the DC power supply (my code for that part was heavily based on a sample I received). While the code seemed to be correctly reading the retention times, we received only about 99.96% accuracy per array (each array is 512 rows of 16 words each - 262144 bits altogether) at zero retention time (meaning we wrote and then read the data with almost no delay between these operations). This was surprising, as we had expected about 100% accuracy at such a low retention time. We did several trials and determined that it was not the same bits failing each time, but rather a random selection. We broadened our ranges of input voltages in hopes that perhaps an ideal combination of input voltages would lead to better results, but 99.96% accuracy was about the closest we could get. Meanwhile, at ideal voltages about 7 bits per word were never failing no matter how long the idle period was. Testing the code using unideal voltages confirmed the code's accuracy via the never-failing bits (because in that situation all bits fail at some point), so we seemed to be facing hardware difficulty. Troubleshooting this difficulty helped me learn more about the inner workings of Lupulus gain cells and the test board used with it. I also gained a deeper understanding of the interaction amongst the transistors in the gain cell. However, we were unfortunately unsuccessful at pinpointing the problem and determining a solution during my time interning at the lab.

I am thankful for the opportunity to learn about gain cells, the Lupulus and Leo2 chips, and the behaviors of transistors. My time at EnICS also gave me a greater appreciation for the scope and capabilities of C, Python, and the interaction between software and hardware in memory chips.